



Due to gemini limitations, to use string search, refer to link for raw logs:
<https://share.gemini.google/Ce21yU4ziY7Q>

InfoVis AI Log

<https://gemini.google.com/share/f6647efdf6cd>



Created with 3.1 Pro 1 July 2026 at 08:58 • Published on 24 July 2026 at 19:45

Currently I have a tibble that contains rows e.g.
 product country quantity
 <chr> <chr> <dbl>
 1 01 Wheat Afghanistan 2.72



Dealing with mixed units in a single column is a classic data wrangling hurdle. Since you want to exclude those specific USD-based products before summing up the tonnage, you just need to add an exclusion condition to your `filter()` step.

The cleanest way to do this in R is by using the `%in%` operator combined with the `!` (NOT) operator.

Here is your updated code:

R



```
# Define the products to exclude for cleaner code
usd_products <- c(
  "18x Live Animals",
  "21 Fruits",
  "22 Vegetables",
  "25 Processed fruits and veget"
)
```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

```

country != "World ",
!(product %in% usd_products) # Drops rows matching the USD products
) |>
group_by(country) |>
summarise(export_quantity = sum(quantity, na.rm = TRUE), .groups = "drop")

```

Why this works:

`%in%` : This checks if the value in your `product` column exists anywhere inside the `usd_products` vector.

! (NOT): By wrapping the condition in `!(...)`, you are telling R to keep only the rows where the product is **not** in that list.

Multiple conditions in `filter()` : You can just separate multiple conditions with a comma, which acts as an implicit "AND" (`&`).

A quick side note: I kept your exact string `"World "` in the first filter condition. If that trailing space was a typo, you might want to change it to `"World"` to ensure you are actually filtering out the global total. If your dataset genuinely has a trailing space there, you are good to go.

Could you help to provide a code to make sure that the values are different?

```
quantity_w32_verify <- quantity_2024 |>
```

✓

Because data frames contain text columns (like `country`) and the row orders might not perfectly match if some countries completely dropped out, you shouldn't directly subtract one data frame from another using `df1 - df2`.

Instead, the safest and most informative way to verify this in R is to **join** the two tables by the `country` column and calculate the difference row-by-row.

Here is the code to do exactly that using `dplyr`:

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

R



```
library(dplyr)

# 1. Create your verification dataset (includes USD products)
quantity_w32_verify <- quantity_2024 |>
  filter(country != "World ") |>
  group_by(country) |>
  summarise(export_quantity_all = sum(quantity, na.rm = TRUE), .groups = "drop")

# 2. Join the two datasets and compare
comparison_check <- quantity_w32_verify |>
  left_join(quantity_w32, by = "country") |>
  mutate(
    # Calculate the exact difference
    difference = export_quantity_all - export_quantity,

    # Check if they are different (using dplyr::near is safer than != 0 for decimal)
    is_different = !near(export_quantity_all, export_quantity)
  ) |>
# 3. Optional: Filter to see ONLY the countries where the values changed
filter(is_different == TRUE)

# View the result
print(comparison_check)
```

Why use this approach?

1. `left_join(..., by = "country")` : This ensures you are comparing the exact same country to itself, even if the sorting or number of rows changed between the two datasets.
2. `near()` **instead of** `!= 0` : Computers handle decimal numbers (like tonnes) a bit weirdly under the hood (known as floating-point precision). `dplyr::near()` is a built-in function that safely checks if two numbers are the same, avoiding false positives caused by microscopic decimal differences.
3. `filter(is_different == TRUE)` : This final step strips away all the countries that

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Currently the output of `quantity_2024` is like so,
`quantity_2024 <- quantity_raw |> select(product =
 "ProductDescription", country = "ReporterName", quantity =
 "2024") |> filter(product != "AoA HS proxy", !(prod` ✓

It is completely understandable why that is frustrating! You are right that tibbles (the default data frame format in the tidyverse) intentionally truncate their console output to the first 10 rows. This is designed to prevent a massive dataset from flooding and freezing your console, but it does mean you cannot just visually scan the output for a specific word.

However, the data is still all there under the hood. You don't need to print the whole table to check for a value; you can ask R to search for it directly.

Here are the best ways to check if "World" exists in your `country` column, depending on what you want to see:

1. The Quickest Way: A simple TRUE or FALSE

If you just want a definitive "Yes, it is there" or "No, it is not," use base R's `%in%` operator or the `any()` function.

R



```
# Checks if "World" exists anywhere in the country column
"World" %in% quantity_2024$country

# Alternatively:
any(quantity_2024$country == "World")
```

If this returns TRUE , the country "World" is in your dataset. If FALSE , it is not.

2. The Tidyverse Way: Filtering

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

If you want to not only know *if* it exists, but also see the actual data for that row (like what product or quantity is associated with it), use `filter()` .

R



```
quantity_2024 |>
  filter(country == "World")
```

If "World" doesn't exist, this will simply return an empty tibble (0 rows). If it does exist, it will print only the rows where the country is "World".

3. The Visual Way: Inspecting all distinct countries

Sometimes "World" might be misspelled, have a trailing space (like "World " from your previous query), or be lowercase. If you want to see a clean list of every unique country in your dataset to double-check manually:

R



```
# Get a list of all unique countries, sorted alphabetically
quantity_2024 |>
  distinct(country) |>
  arrange(country) |>
  print(n = Inf) # This overrides the 10-row limit and prints everything
```

4. RStudio GUI (Bonus)

If you are using RStudio, you can always pop open the dataset in a spreadsheet-like viewer by running:

Save this chat

R



Gemini may display inaccurate info, including about people, so double-check its responses.

View(quantity_2024)

From there, you can use the search bar in the top right of the viewer pane to type in "World".

Currently, there are countries with differences and no differences after filtering out the 4 product categories. However, is it possible to also show the 4 values of the categories we removed to ensure that it adds up t



Yes, you can absolutely do that! It is actually a fantastic practice to build in this kind of rigorous "audit trail" when wrangling data.

To achieve this, we need to isolate those 4 USD products from your raw data, flip them into their own columns (so you can see them side-by-side for each country), and then join that back to your difference check.

Before we jump into the new code, **I noticed a tiny typo in your current code** that will cause your `difference` column to always be `0`. In your `mutate()` step, you have `export_quantity_verify - export_quantity_verify`. You will want to change that to `export_quantity_check - export_quantity_verify`.

Here is how you can build that complete verification table:

R



```
library(dplyr)
library(tidyr) # We need this for pivot_wider

usd_products <- c("18x Live Animals", "21 Fruits", "22 Vegetables", "25 Processed")

# 1. First, fix the typo in your comparison check
comparison_check <- quantity_w32_verify |>
  left_join(quantity_2024_check, by = "country") |>
  mutate(
    # Fixed the typo here:
    difference = export_quantity_check - export_quantity_verify,
```

Gemini may display inaccurate info, including about people, so double-check its responses.

```

comparison_check_difference <- comparison_check |>
  filter(is_different == TRUE)

# 2. Create the breakdown of the 4 removed products
usd_breakdown <- quantity_raw |>
  select(
    product = "ProductDescription",
    country = "ReporterName",
    quantity = "2024"
  ) |>
  # Keep ONLY the 4 USD products we want to inspect
  filter(country != "World", product %in% usd_products) |>
  # Group by both country and product in case there are multiple rows per product
  group_by(country, product) |>
  summarise(quantity = sum(quantity, na.rm = TRUE), .groups = "drop") |>
  # Pivot the products into their own columns for easy reading
  pivot_wider(
    names_from = product,
    values_from = quantity,
    values_fill = 0 # If a country didn't trade a specific product, put a 0 instead
  ) |>
  # Sum the 4 new columns to ensure it perfectly matches your "difference" column
  mutate(
    sum_of_removed = rowSums(across(any_of(usd_products)), na.rm = TRUE)
  )

# 3. Join this breakdown back to your countries that had a difference
final_audit_trail <- comparison_check_difference |>
  left_join(usd_breakdown, by = "country")

# View the result
print(final_audit_trail)

```

What is happening here?

`pivot_wider()` : This is the magic function for this task. It takes your dataset (which is currently "long" with a single `product` column) and makes it "wide" by creating a brand new column for each unique product name.

`rowSums(across(any_of(...)))` : This safely adds up all the values across the 4 newly created product columns for each row, giving you a `sum_of_removed` column.

When you look at `final_audit_trail`, your `difference` column should be mathematically identical to the new `sum_of_removed` column, and you will be able to see exactly which specific products contributed to that number!

Currently I want to have a "Rest of the region" data for each region, where I have selected the top 3 from each region. This is how I have currently mutated the data,



Because `slice_max()` works by completely filtering out the rows that don't make the cut, those "Rest of the region" values are getting dropped before you can add them up.

To keep that data and group it into a new category, we need to shift strategies. Instead of filtering, we can use the incredibly handy `fct_lump_n()` function from the **forcats** package (which is part of the core tidyverse). It is explicitly designed to lump the smallest categories into an "Other" category based on a weighting variable.

Here is how you can seamlessly integrate it into your pipeline:

R



```
library(dplyr)
library(tidyr)
library(forcats) # Make sure forcats is loaded

regional_data <- processed_data |>
  # 1. Group by region to ensure we evaluate the top 3 WITHIN each region
  group_by(geo_region) |>

  # 2. Reclassify the country column (assuming it's called 'country')
  mutate(
    country_grouped = fct_lump_n(
      country, n = 3, other_label = "Rest of the region"
    )
  )
```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

```

    other_level = "Rest of Region"
  )
) |>

```

3. Group by the new classification to sum up the "Rest of Region" values

```

group_by(geo_region, country_grouped) |>
summarise(
  export_value = sum(export_value, na.rm = TRUE),
  export_quantity = sum(export_quantity, na.rm = TRUE),
  .groups = "drop"
) |>

```

4. Apply your exact pivot and formatting logic

```

pivot_longer(
  cols = c(export_value, export_quantity),
  names_to = "metric_type",
  values_to = "metric_value"
) |>
mutate(
  metric_type = case_when(
    metric_type == "export_value" ~ "Export Value (USD Millions)",
    metric_type == "export_quantity" ~ "Export Quantity (1,000 Tonnes)"
  )
)

```

Why this works:

`fct_lump_n()` : This evaluates your `country` column. It looks at the weight (`w = export_value`), keeps the top 3 (`n = 3`) unique country names exactly as they are, and renames every other country in that region to "Rest of Region" .

The Second `group_by()` : By grouping by both the region and our newly lumped country column, our `summarise()` step perfectly aggregates all the smaller countries into a single "Rest of Region" row per region.

Preserved Metrics: Because we sum up both `export_value` and `export_quantity` before pivoting, your "Rest of Region" row will accurately reflect the totals for both metrics.

(Note: Just ensure your actual country column name is used in the `f = country` argument if it is named something else, like `ReporterName`)

Gemini may display inaccurate info, including about people, so double-check its responses.

Do you plan to plot this data next, and if so, do you want to ensure "Rest of Region" is always forced to the bottom of the bars or legends?

At the moment, I have created a faceted lollipop plot, however my end goal is actually a diverging lollipop plot, where e.g. the export quantity is on the left, while the export value is on the right, how do I achieve this? ✓

To create a true diverging lollipop plot (often called a butterfly chart) when your two variables have entirely different units and scales (Tonnes vs. USD), you run into a limitation with `facet_wrap`. If you try the old-school trick of multiplying one metric by `-1`, they are forced to share the same x-axis scale, which will visually crush whichever metric has smaller numbers.

The absolute cleanest way to achieve this in R is by using the `patchwork` package. It allows you to build two separate plots with their own independent scales and stitch them together seamlessly, placing the country names perfectly in the middle.

Here is how to modify your approach to get that diverging effect.

The Solution: Using `patchwork`

If you don't have it installed already, run `install.packages("patchwork")`.

R



```
library(ggplot2)
library(dplyr)
library(patchwork) # The magic package for stitching plots
```

```
# 1. Split the data into two separate dataframes
data_quantity <- global_top30 |>
  filter(metric_type == "Export Quantity (1,000 Tonnes)")
```

```
data_value <- global_top30 |>
```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

```
# We reverse the X-axis and move the Y-axis labels to the right (the middle)
p_left <- ggplot(data_quantity, aes(x = metric_value, y = display_entity)) +
  geom_segment(
    aes(x = 0, xend = metric_value, y = display_entity, yend = display_entity),
    color = "gray80", linewidth = 0.8
  ) +
  geom_point(aes(color = geo_region), size = 3) +
  scale_x_reverse(labels = scales::comma) + # This makes the lollipops grow to t
  scale_y_discrete(position = "right") +   # Puts the country names on the righ
  theme_minimal() +
  theme(
    legend.position = "none", # Hide legend here to avoid duplicates
    panel.grid.minor = element_blank(),
    panel.grid.major.y = element_blank(),
    axis.title.y = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold", size = 10)
  ) +
  labs(
    title = "Export Quantity (1,000 Tonnes)",
    x = NULL
  )
```

3. Build the Right Plot (Value)

Standard orientation, but we hide the Y-axis text so it doesn't duplicate

```
p_right <- ggplot(data_value, aes(x = metric_value, y = display_entity)) +
  geom_segment(
    aes(x = 0, xend = metric_value, y = display_entity, yend = display_entity),
    color = "gray80", linewidth = 0.8
  ) +
  geom_point(aes(color = geo_region), size = 3) +
  scale_x_continuous(labels = scales::comma) +
  theme_minimal() +
  theme(
    legend.position = "bottom", # Keep the legend on this plot
    panel.grid.minor = element_blank(),
    panel.grid.major.y = element_blank(),
    axis.text.y = element_blank(), # Hide country names (they are on the left p
    axis.title.y = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold", size = 10)
  ) +
  labs(
    title = "Export Value (USD Millions)",
    x = NULL,
    color = "Geographic Region"
```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

```
# 4. Stitch them together using patchwork
final_plot <- p_left + p_right +
  plot_layout(guides = "collect") + # Gathers the legend nicely
  plot_annotation(
    title = "Value vs. Volume: Top 30 Global Agricultural Exporters (2024)",
    theme = theme(plot.title = element_text(hjust = 0.5, face = "bold", size = 1
  )

# Display the plot
final_plot
```

Why this works:

`scale_x_reverse()` : This is applied to `p_left` . It keeps your numbers positive in the underlying data but visually flips the axis so the lollipops grow from right to left.

`scale_y_discrete(position = "right")` : This moves the country names to the right side of `p_left` . When you stick `p_right` next to it, the country names form a perfect spine down the center of your chart.

Independent Axes: Because `p_left` and `p_right` are technically two separate ggplot objects, they independently calculate their own optimal x-axis scales, ensuring neither the tonnage nor the USD values are squished.

`plot_layout(guides = "collect")` : This tells patchwork to look for legends across all plots and consolidate them into a single, clean legend at the bottom (as defined in `p_right` 's theme).

Currently we only have the region on the right plot, however, "`plot_layout(guides = "collect")`" takes up too much space on the right, is there a `plot_layout` which puts the guides at the bottom instead? ✓

These are great formatting refinements! When you stitch plots together with `patchwork` , the combined plot actually develops its own "global" theme that sits on top of the

1. Moving the Collected Legend to the Bottom

When you use `plot_layout(guides = "collect")`, patchwork strips the legend from your individual plots and creates a new global legend. By default, it sticks this global legend on the right.

To override this, you use the `&` operator. In `patchwork`, adding `& theme(...)` at the very end applies that theme to the *entire combined plot*, including the newly collected legend.

2. Center-Aligning the Y-Axis Text

Assuming you kept the Y-axis labels attached to your left plot (`p_left`) using `scale_y_discrete(position = "right")`, ggplot defaults to aligning the text flush against the axis line. You can force it to center by adjusting the horizontal justification (`hjust = 0.5`) in the `theme()` of whichever plot is displaying the labels.

Here is how those specific parts of your code should be updated:

R



```
# UPDATE 1: Add this to the theme() of whichever plot holds your country names (
p_left <- ggplot(...) +
  # ... [your geoms and scales] ...
  theme(
    # Center-align the text
    axis.text.y = element_text(hjust = 0.5),

    # Hide the legend for the individual plot (as before)
    legend.position = "none",

    # ... [rest of your theme settings] ...
  )

# UPDATE 2: Use the `&` operator when stitching them together
final_plot <- p_left + p_right +
  plot_layout(guides = "collect")
```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

```
) & # Note the & operator here!  
theme(  
  # This forces the globally collected legend to the bottom  
  legend.position = "bottom"  
)  
  
final_plot
```

A quick tip on centering Y-axis text:

If you notice that `hjust = 0.5` doesn't seem to perfectly center the text, it is because `ggplot` gives the text block a width exactly equal to the longest country name. If you want more breathing room around the text so the centering is more obvious, you can add a fixed width to the labels using the `stringr` package inside your axis scale:

```
scale_y_discrete(position = "right", labels = function(x)  
  stringr::str_pad(x, width = 20, side = "both"))
```

I'm considering moving the x-axis to the top as the title are Export Values (\$USD Million) and Export Quantity. However, I am contemplating whether it is appropriate for lollipop charts

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Moving the x-axis to the top is not just appropriate—it is actually considered a **best practice** for the exact type of chart you are building!

When you have a long list of categories (like 30 countries) reading from top to bottom, forcing the viewer to scroll all the way down to the bottom of the page just to understand the scale causes visual friction. By placing the x-axis at the top, you give them the "legend" for the scale immediately, before they start scanning the data.

Because a lollipop chart is essentially just a thinner bar chart, all the same rules apply. It will look incredibly clean.

How to do it in your code

Since you are already using `patchwork` and have two separate plots, it is remarkably easy to flip the axes to the top. You just need to add `position = "top"` inside your existing `scale_x_*` functions.

Here is what you need to change in your `p_left` and `p_right` code:

For the Left Plot (Quantity):

R



```
p_left <- ggplot(data_quantity, aes(x = metric_value, y = display_entity)) +
  # ... [your geoms stay the same] ...

  # Add position = "top" here
  scale_x_reverse(labels = scales::comma, position = "top") +

  # ... [the rest of your scales and themes]
```

For the Right Plot (Value):

R



```
p_right <- ggplot(data_value, aes(x = metric_value, y = display_entity)) +
```

Gemini may display inaccurate info, including about people, so double-check its responses.

```
# Add position = "top" here
scale_x_continuous(labels = scales::comma, position = "top") +

# ... [the rest of your scales and themes]
```

A Quick Design Consideration

If you move the axis to the top, the numbers will now sit right below the individual plot titles you set earlier (`title = "Export Value (USD Millions)"`). This can look a bit cluttered.

The cleaner approach: 1. Remove the `title` argument from your `labs()` in both plots.
2. Put those titles directly into the axis name instead.

You can do this seamlessly inside the scale function:

R



```
scale_x_continuous(
  name = "Export Value (USD Millions)", # This replaces the plot title
  labels = scales::comma,
  position = "top"
)
```

This ensures the label acts as both the title of that specific column *and* the axis label, keeping your design tight and professional.

How are the country names in the middle looking—do you feel they need more breathing room between the left and right lollipops?

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

I decided to add `geom_text()` to display the values however, some of it has been clipped (mostly Brazil and EU).

```
``{r}
```



This is a very common formatting quirk in `ggplot2` ! By default, `ggplot` calculates the boundaries of your x-axis based *only* on the actual data points (the dots at the end of your lollipops). It doesn't take into account the extra physical space required by the text labels you just added.

Because Brazil and the EU are your top exporters, their dots sit right at the very edge of the plot. When you push the text just past those dots using `hjust`, it falls outside the "canvas" and gets clipped.

The Solution: Expanding the X-Axis Limits

You don't need to hardcode specific axis limits. Instead, you can use the `expand` argument inside your `scale_x_*` functions to tell `ggplot` to automatically add a little bit of extra "breathing room" (padding) to the edges of the plot.

Here is how you update your scales to fix the clipping.

For `p_left` (Quantity):

Because you reversed this axis, the larger numbers are on the *left*. We need to add padding to the "upper" end of the reversed scale.

R



```
# Update this line in p_left
scale_x_reverse(
  labels = scales::comma,
  position = "top",
  # expand takes a multiplier: c(padding_at_zero, padding_at_max)
```

Save this chat

```
    expand = expansion(mult = c(0.05, 0.15)) # Adds 15% extra space to the left
  ) +
```

For p_right (Value):

This is a standard axis, so the larger numbers are on the *right*.

R



```
# Update this line in p_right
scale_x_continuous(
  labels = scales::comma,
  position = "top",
  expand = expansion(mult = c(0.05, 0.15)) # Adds 15% extra space to the right
) +
```

Putting it together

Here is exactly how those changes slot into your existing, excellent code:

R



```
p_left <- ggplot(data_quantity, aes(x = metric_value, y = display_entity)) +
  geom_segment(
    aes(x = 0, xend = metric_value, y = display_entity, yend = display_entity),
    color = "gray80", linewidth = 0.8
  ) +
  geom_point(aes(color = geo_region), size = 3) +
  geom_text(
    aes(label = scales::comma(metric_value, accuracy = 1)),
    hjust = 1.3, size = 2.5, color = "gray30"
  ) +
  # Added the expand argument here:
  scale_x_reverse(labels = scales::comma, position = "top", expand = expansion(mult = c(0.05, 0.15))) +
  scale_y_discrete(position = "right") +
```

Gemini may display inaccurate info, including about people, so double-check its responses.

```

    legend.position = "none",
    panel.grid.minor = element_blank(),
    panel.grid.major.y = element_blank(),
    axis.title.y = element_blank(),
    axis.text.y = element_text(hjust = 0.5),
    plot.title = element_text(hjust = 0.5, face = "bold", size = 10)
  ) +
  labs(
    title = "Export Quantity (1,000 Tonnes)",
    x = NULL,
    color = "Region"
  )

p_right <- ggplot(data_value, aes(x = metric_value, y = display_entity)) +
  geom_segment(
    aes(x = 0, xend = metric_value, y = display_entity, yend = display_entity),
    color = "gray80", linewidth = 0.8
  ) +
  geom_point(aes(color = geo_region), size = 3) +
  geom_text(
    aes(label = scales::comma(metric_value, accuracy = 1)),
    hjust = -0.3, size = 2.5, color = "gray30"
  ) +
  # Added the expand argument here:
  scale_x_continuous(labels = scales::comma, position = "top", expand = expansion(margin = 1)) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    panel.grid.minor = element_blank(),
    panel.grid.major.y = element_blank(),
    axis.text.y = element_blank(),
    axis.title.y = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold", size = 10)
  ) +
  labs(
    title = "Export Value ($USD Millions)",
    x = NULL,
    color = "Region"
  )

# The patchwork stitching remains exactly the same
final_plot <- p_left + p_right +
  plot_layout(guides = "collect") +
  plot_annotation(

```

Gemini may display inaccurate info, including about people, so double-check its responses.

```

    theme = theme(plot.title = element_text(hjust = 0.5, face = "bold", size
) &
theme(legend.position = "bottom") &
scale_color_brewer(palette = "Dark2")

final_plot

```

A quick tip on `expansion()`

If `0.15` (15% padding) feels like it leaves *too* much blank space at the edges, just bump those numbers down to something like `0.10` or `0.08` until the margins look perfectly snug around your text.

Did expanding the limits f I am trying to highlight outliers in my plot, and currently I am bolding and coloring my outlier vector with `element_text()`

```
axis.text.y = element_text(hjust = 0.5, face =
```



That warning appears because the creators of `ggplot2` intentionally do not support vectorizing `element_text()`. While it might technically render right now, it is a fragile hack that will likely break your code in a future package update.

The cleanest, officially supported, and most robust way to conditionally format individual axis labels in R is to use the `ggtext` package. It allows you to use HTML and Markdown directly inside your ggplot labels.

Here is how you can completely replace your vectorized `element_text()` approach without changing your underlying dataframe.

The Fix: Using `ggtext` and Named Vectors

If you don't have it installed, you will first need to run `install.packages("ggtext")`.

Save this chat

R



Gemini may display inaccurate info, including about people, so double-check its responses.


```

library(ggplot2)
library(dplyr)
library(ggtext) # Enables HTML/Markdown in ggplot text

# 1. Pre-compute your formatted labels as a named vector
# We wrap the text in simple HTML tags to dictate bolding and color
formatted_labels <- case_when(
  levels(global_top30$display_entity) %in% high_value ~ paste0("<b style='color:"
  levels(global_top30$display_entity) %in% low_value ~ paste0("<b style='color:"
  .default = paste0("<span style='color:gray30;'>", levels(global_top30$display_entity))
)

# Crucial step: Name the vector so ggplot maps the correct label to the correct
names(formatted_labels) <- levels(global_top30$display_entity)

# 2. Build your plot (showing p_left as the example)
p_left <- ggplot(data_quantity, aes(x = metric_value, y = display_entity)) +
  # ... [your geoms remain exactly the same] ...

  # Pass your new vector to the labels argument here
  scale_y_discrete(position = "right", labels = formatted_labels) +
  theme_minimal() +
  theme(
    # Replace element_text with element_markdown so it parses the HTML!
    axis.text.y = element_markdown(hjust = 0.5),

    # ... [the rest of your theme settings] ...
  )

```

Why this is the standard approach:

No ggplot2 warnings: You are passing a static set of labels to the scale rather than forcing the theme function to evaluate conditional logic on the fly.

Separation of concerns: `scale_y_discrete()` handles *what* the text is (including the HTML styling), and `theme(axis.text.y = element_markdown())` simply acts as the engine to render it.

Perfect matching: By setting `names(formatted_labels) <- levels(...)`, you guarantee that even if ggplot reorders the axis, the correct styling will stick.

Gemini may display inaccurate info, including about people, so double-check its responses.

It seemed that when I changed to use `element_markdown`, the text in the plot, those that are to be colored are not colored and are instead, e.g. `<b style='color:#B2182B'> Argentina </b>`

Whenever you see literal HTML tags printed on your plot like `<b style='color:#B2182B'> Argentina </b>`, it means exactly one thing: `ggplot2` is still treating that element as standard `element_text()` rather than switching over to `ggtext`'s `element_markdown()`.

There are two common culprits for this when building a plot, especially with right-aligned axes. Here is how to fix them both.

Culprit 1: Theme Order (The most common trap)

In `ggplot2`, themes overwrite each other based on order. If you put `theme_minimal()` *after* your custom `theme()` block, `theme_minimal()` will instantly erase your `element_markdown()` setting and reset the axis back to standard text.

The Fix: Always ensure `theme_minimal()` comes *before* your custom `theme(...)` block.

Culprit 2: The "Right Axis" Quirk

Because you moved the Y-axis to the right side using `scale_y_discrete(position = "right")`, `ggplot2` sometimes looks for a highly specific theme element called `axis.text.y.right`. If it doesn't find it, it defaults back to standard text.

The Fix: Explicitly set both `axis.text.y` and `axis.text.y.right` to `element_markdown()`.

The Corrected Code Snippet

Save this chat

Here is exactly how the bottom half of your `plot_left` code should look to guarantee the

Gemini may display inaccurate info, including about people, so double-check its responses.

R



```
p_left <- ggplot(data_quantity, aes(x = metric_value, y = display_entity)) +
  # ... [your geom_segment, geom_point, geom_text, and scale_x_reverse stay ex

# 1. Pass your formatted labels
scale_y_discrete(position = "right", labels = formatted_labels) +

# 2. Base theme MUST come before custom theme
theme_minimal() +

# 3. Custom theme applies your markdown
theme(
  legend.position = "none",
  panel.grid.minor = element_blank(),
  panel.grid.major.y = element_blank(),
  axis.title.y = element_blank(),

  # FIX: Target both the general y-axis AND the right-side y-axis specific
  axis.text.y = element_markdown(hjust = 0.5),
  axis.text.y.right = element_markdown(hjust = 0.5),

  plot.title = element_text(hjust = 0.5, face = "bold", size = 10)
) +
labs(
  title = "Export Quantity (1,000 Tonnes)",
  x = NULL,
  color = "Region"
)
```

Make sure you run your `library(ggtext)` at the top of your script as well. Once you update that specific block in `p_left` (making sure `theme_minimal()` is on top and both `axis.text.y` and `axis.text.y.right` are targeted), the raw HTML tags will vanish and your custom colors and bolding will appear perfectly in the center!

Currently I am using patchwork to create this diverging
lollipop plot, and I am thinking of creating a diverging lollipop

Gemini may display inaccurate info, including about people, so double-check its responses.

faceted layout perhaps) with the data I have transformed previously. Is this achievable without patchwork?

Yes, it is absolutely achievable in a single `ggplot2` call without `patchwork` !

To do this, we use a classic data visualization math trick: **we temporarily make the Export Quantity values negative in the data.**

Because `ggplot2` draws axes from negative to positive (left to right), a negative value starting at 0 will naturally draw a lollipop pointing to the left. We then use a custom label function to strip away the minus sign so the viewer only sees the positive numbers.

Here is the exact code to achieve this faceted layout:

The "Negative Value" Facet Strategy

R



```
library(ggplot2)
library(dplyr)

# 1. Prep the data: Convert Quantity to negative and lock column order
regional_data_plot <- regional_data |>
  mutate(
    # Ensure Quantity is the first level so it renders on the left column
    metric_type = factor(
      metric_type,
      levels = c("Export Quantity (1,000 Tonnes)", "Export Value (USD Millions)"
    ),

    # The Trick: Make quantity negative so lollipops point left
    plot_value = if_else(
      metric_type == "Export Quantity (1,000 Tonnes)",
      -metric_value,
      metric_value
    )
  )
```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

```

ggplot(regional_data_plot, aes(x = plot_value, y = country_grouped)) +
  geom_segment(
    aes(x = 0, xend = plot_value, y = country_grouped, yend = country_grouped),
    color = "gray80", linewidth = 0.8
  ) +
  geom_point(aes(color = geo_region), size = 3) +
  geom_text(
    # Use abs() to ensure the text labels don't show the negative sign
    aes(
      label = scales::comma(abs(plot_value), accuracy = 1),
      # Conditionally push text outward based on whether it's left or right
      hjust = if_else(metric_type == "Export Quantity (1,000 Tonnes)", 1.3, -0.3
    ),
    size = 2.5, color = "gray30"
  ) +
  # Facet by BOTH Region and Metric
  facet_wrap(
    ~ geo_region + metric_type,
    ncol = 2,          # Forces 2 columns: Left = Quantity, Right = Value
    scales = "free"    # Crucial: Allows independent x-axes AND independent y-axes
  ) +
  scale_x_continuous(
    position = "top",
    # The Second Trick: Strip the negative sign from the axis labels using abs()
    labels = function(x) scales::comma(abs(x)),
    expand = expansion(mult = c(0.2, 0.2)) # Breathing room for the text labels
  ) +
  theme_minimal() +
  theme(
    legend.position = "none", # Hide legend since regions are already facet headers
    panel.grid.minor = element_blank(),
    panel.grid.major.y = element_blank(),
    axis.title.y = element_blank(),

    # Format the facet headers for clarity
    strip.text = element_text(face = "bold", size = 10),
    panel.spacing.x = unit(1, "lines"),
    panel.spacing.y = unit(1.5, "lines")
  ) +
  labs(
    title = "Regional Leaders: Value vs. Volume",
    x = NULL
  )

```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

The One Trade-off

This approach is significantly less code than looping through `patchwork`, but there is one visual compromise: **the country names will appear on the far-left of each row**, rather than forming a perfect "spine" down the center between the two plots.

In a standard `ggplot2` facet, the Y-axis is locked to the left side of the grid. If you strongly prefer the "center spine" look for every single region, `patchwork` remains the only way to perfectly stitch two distinct plots together with a central axis.

Does placing the country labels on the left work for your layout, or is the center spine a strict requirement for the final design?

Isn't this negative trick not recommended as our two metrics have different units and magnitudes? Forcing them on the same x-axis may be misleading

You are absolutely right to be cautious here—mapping two entirely different units and magnitudes onto a single shared scale is a cardinal sin in data visualization.

However, in the faceted code above, they actually **do not share the same axis limits**.

Because we included `scales = "free"` inside `facet_wrap()`, `ggplot` calculates a completely independent mathematical x-axis for every single panel.

Here is what happens under the hood:

The Quantity panel evaluates its own data and draws an axis from, say, 0 to -50,000 (tonnes). The zero-line sits on the far right of its panel.

The Value panel evaluates its own data and draws an axis from 0 to 5,000 (USD). The zero-line sits on the far left of its panel.

When placed side-by-side, the two zero-lines meet in the middle, creating a **Butterfly Chart**. Butterfly charts are the standard industry exception for diverging plots with different units, *provided* the units are explicitly labeled at the top of each side (which our facet headers handle). It allows viewers to compare rank and relative magnitude across

That said, your intuition about your audience matters most. If you suspect your stakeholders will misinterpret the opposing directions as a shared scale despite the labels, the safest approach is to ditch the negative trick entirely. By keeping all values positive, both facets will draw lollipops from left to right, allowing the viewer to read the chart like a traditional book.

Would you prefer to drop the negative trick and keep both metrics pointing right in a standard facet, or would you like to see how to loop `patchwork` to generate perfect center-aligned butterfly charts for each individual region?

Currently this is how I previously processed the data,

```
regional_data <- processed_data |>
  group_by(geo_region) |>
```



Because `fct_lump_n()` only accepts a single, fixed string for its `other_level` argument, it will always output the exact same phrase for every region.

To make it dynamic based on the region, we need to modify the data directly after the lumping step. The safest way to do this is to temporarily convert the lumped factor into a character string, use `if_else()` to append the region name, and then proceed with your aggregation.

Since this changes the name of your "Rest of" categories, we also need to slightly update your plot prep logic so your `is_rest` column can still identify them correctly for sorting.

Here are the two updated code blocks to achieve this:

1. Updating `regional_data`

R



```
regional_data <- processed_data |>
  group_by(geo_region) |>
```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

```

    f = country,
    n = 3,
    w = export_value,
    other_level = "Rest of Region"
  ),

# 2. Convert from factor to character to avoid "invalid factor level" warning
display_entity = as.character(display_entity),

# 3. Swap out the generic phrase with the region-specific phrase
display_entity = if_else(
  display_entity == "Rest of Region",
  paste("Rest of", geo_region),
  display_entity
)
) |>
# 4. Group by your new dynamically named entities and summarize
group_by(geo_region, display_entity) |>
summarise(
  export_value = sum(export_value, na.rm = TRUE),
  export_quantity = sum(export_quantity, na.rm = TRUE),
  .groups = "drop"
) |>
pivot_longer(
  cols = c(export_value, export_quantity),
  names_to = "metric_type",
  values_to = "metric_value"
) |>
mutate(
  metric_type = case_when(
    metric_type == "export_value" ~ "Export Value (USD Millions)",
    metric_type == "export_quantity" ~ "Export Quantity (1,000 Tonnes)"
  )
)

```

2. Updating the Plot Prep logic

Since your "Rest of" categories are now dynamically named (e.g., "Rest of Asia", "Rest of Europe"), `display_entity == "Rest of Region"` will no longer work. We can swap that exact match check for `startsWith()`.

Gemini may display inaccurate info, including about people, so double-check its responses.

R



```
regional_plot_data <- regional_data |>
  mutate(entity_key = paste(geo_region, display_entity, sep = "___"))

key_levels <- regional_plot_data |>
  filter(metric_type == "Export Value (USD Millions)") |>

# UPDATE: Check if the string starts with "Rest of" instead of an exact match
mutate(is_rest = startsWith(display_entity, "Rest of")) |>

arrange(geo_region, desc(is_rest), metric_value) |>
pull(entity_key)

regional_plot_data <- regional_plot_data |>
  mutate(entity_key = factor(entity_key, levels = key_levels))

data_qty <- filter(regional_plot_data, metric_type == "Export Quantity (1,000 Tc
data_val <- filter(regional_plot_data, metric_type == "Export Value (USD Millior
```

Why startsWith() ?

By using `startsWith(display_entity, "Rest of")`, R will instantly flag any row that begins with those words as `TRUE`, safely identifying your grouped entities regardless of which region is tacked onto the end of the string. This preserves your `arrange(geo_region, desc(is_rest), metric_value)` logic, ensuring the "Rest of X" categories are always pinned to the bottom of their respective region groups!

How do facet_grids work?

While `facet_wrap()` takes a long sequence of plots and wraps them into a layout (like words wrapping to the next line in a document), `facet_grid()` builds a strict **2D matrix** based on your variables—exactly like a pivot table.

View this chat

grid.

The Syntax

The magic of `facet_grid()` lies in the tilde (`~`) formula, which dictates exactly which variable controls the rows and which controls the columns:

`facet_grid(row_variable ~ column_variable)` : Creates a full grid crossing both variables.

`facet_grid(. ~ column_variable)` : Creates a single row of panels side-by-side (the `.` means "nothing on the rows").

`facet_grid(row_variable ~ .)` : Creates a single vertical column of panels stacked on top of each other.

Note: In modern `ggplot2`, it is highly recommended to use the `vars()` function instead of the tilde formula (e.g., `facet_grid(rows = vars(geo_region), cols = vars(metric_type))`). It is more readable and less prone to breaking when programming.

The Superpowers

For grouped categorical data, `facet_grid()` unlocks a few abilities that `facet_wrap()` cannot do.

1. Proportional Panel Sizing (`space = "free"`)

By default, all facet panels are drawn at the exact same physical size. Imagine you didn't restrict your regions to just the top 3 countries—if Europe had 15 countries and Oceania had 2, the Oceania panel would stretch its lollipops to be massive, while Europe's would be squished together to fit the same height.

If you use `facet_grid(rows = vars(geo_region), scales = "free_y", space = "free_y")`, `ggplot2` dynamically shrinks and grows the physical height of each panel based on how many categories (countries) are inside it. The lollipops stay the exact same thickness across the entire plot.

[Save this chat](#)

Gemini may display inaccurate info, including about people, so double-check its responses.

If you are doing data exploration, `facet_grid()` has a `margins = TRUE` argument. This automatically adds an extra row and column to your grid that plots the aggregate total of everything across those variables.

Example in Your Context

If you wanted to ditch `patchwork` and plot a perfect grid where rows are your regions and columns are your metrics (Quantity vs. Value), the facet layer would look like this:

R

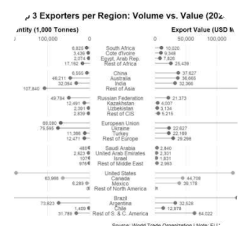


```
ggplot(regional_plot_data, aes(x = metric_value, y = display_entity)) +
  # ... [your geoms] ...
  facet_grid(
    rows = vars(geo_region),
    cols = vars(metric_type),
    scales = "free"
  )
```

Because of `scales = "free"`, the X-axis for Quantity is allowed to be mathematically independent from the X-axis for Value, and each region's Y-axis will only display the countries relevant to that region.

Want to apply this to your regional chart?

↪ Yes



Save this chat

This is what I have brought over from the global top 30 plot

Gemini may display inaccurate info, including about people, so double-check its responses.

This plot is looking exceptionally clean! You have perfectly executed the `patchwork` center-spine alignment, and using `facet_grid()` with `space = "free_y"` successfully grouped the regions without needing messy y-axis breaks.

Looking closely at your provided "image_dc262a.png", there are just a few design refinements you could add to take this from a great chart to a publication-ready one:

1. Increase the Vertical Gaps Between Regions

Right now, the regions stack fairly close to one another. Because you successfully hid the facet strip labels (`strip.text = element_blank()`), the only thing separating the regions is a small default margin.

You can make the groupings much more obvious by increasing the vertical space between the facet panels.

The fix: Add `panel.spacing.y = unit(1.5, "lines")` to the `theme()` of both `p_left` and `p_right`.

2. De-emphasize the "Rest of" Categories

Currently, the "Rest of [Region]" rows carry the exact same visual weight (bold colors, same dot size, same line thickness) as the Top 3 individual countries. Since these are aggregates, they technically represent a different level of data.

The fix: You can map the `alpha` (transparency) aesthetic to your `is_rest` variable so that the Top 3 countries are 100% opaque, but the "Rest of" rows are faded to 40% or 50% opacity. This pushes the aggregates into the background and makes the specific country leaders pop.

3. Fix the "Squished" Text on Zero/Near-Zero Values

In the image, the label for **Rest of North America** on the left plot is overlapping its own dot because the value is essentially 0. When a segment has no length, `hjust` struggles to

The fix: Instead of using `hjust = 1.3` in your `geom_text()`, try using `nudge_x`. Nudging pushes the text a fixed distance in data coordinates regardless of the dot's position. For example, on the left plot, removing `hjust` and using `nudge_x = max(data_qty$metric_value) * 0.05` ensures the text is consistently pushed outward without overlapping the zero line.

Would you like me to provide the specific code adjustments to fade out those "Rest of" categories and tweak the label nudging?

Could you address the "De-emphasize"?

To de-emphasize the "Rest of" categories, we need to map the `alpha` (transparency) aesthetic to those specific rows. This will make the Top 3 countries fully opaque (100%), while fading the grouped regional rows to a softer opacity (e.g., 40%), drawing the viewer's eye directly to the individual leaders.

To do this, we just need to ensure an `is_rest` column exists in your plotting data, and then map it inside your geoms.

1. Add the `is_rest` flag to your main data

In your previous data prep, `is_rest` was only created temporarily inside `key_levels`. Add it to your main `regional_plot_data` right before you split it:

R



```
regional_plot_data <- regional_plot_data |>
  mutate(
    entity_key = factor(entity_key, levels = key_levels),
    # Add a TRUE/FALSE flag for the plot to read
    is_rest = startsWith(as.character(display_entity), "Rest of")
  )
```

```
data_qty <- filter(regional_plot_data, metric_type == "Export Quantity (1,000 Tc
data_val <- filter(regional_plot_data, metric_type == "Export Value (USD Millior
```

Gemini may display inaccurate info, including about people, so double-check its responses.

2. Update the Plot Code

Now, you just add `alpha = is_rest` inside the `aes()` for both your segments and points, and control the exact transparency levels using `scale_alpha_manual()`.

Here is exactly how to update `p_left` and `p_right`:

For `p_left`:

R



```
p_left <- ggplot(data_qty, aes(x = metric_value, y = entity_key)) +
  # Add alpha = is_rest to both segment and point
  geom_segment(aes(xend = 0, yend = entity_key, color = geo_region, alpha = is_r
  geom_point(aes(color = geo_region, alpha = is_rest), size = 2.5) +
  geom_text(
    aes(label = scales::comma(metric_value, accuracy = 1)),
    hjust = 1.3, size = 2.5, color = "gray30"
  ) +
  scale_x_reverse(
    labels = scales::comma,
    position = "top",
    expand = expansion(mult = c(0.25, 0))
  ) +
  scale_y_discrete(position = "right", labels = strip_prefix) +

# Tell ggplot exactly what opacities to use: FALSE (not rest) = 1, TRUE (is re
scale_alpha_manual(values = c("FALSE" = 1, "TRUE" = 0.4)) +

facet_grid(geo_region ~ ., scales = "free_y", space = "free_y") +
labs(title = "Export Quantity (1,000 Tonnes)", x = NULL) +
theme_minimal() +
theme(
  legend.position = "none",
  strip.text = element_blank(),
  axis.title.y = element_blank(),
  axis.text.y.right = element_text(hjust = 0.5),
  panel.grid.major.y = element_blank(),
  panel.grid.minor = element_blank())
```

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

```
    plot.title = element_text(hjust = 0.5, face = "bold", size = 10)
  )
}
```

For p_right :

R



```
p_right <- ggplot(data_val, aes(x = metric_value, y = entity_key)) +
  # Add alpha = is_rest here as well
  geom_segment(aes(xend = 0, yend = entity_key, color = geo_region, alpha = is_r
  geom_point(aes(color = geo_region, alpha = is_rest), size = 2.5) +
  geom_text(
    aes(label = scales::comma(metric_value, accuracy = 1)),
    hjust = -0.3, size = 2.5, color = "gray30"
  ) +
  scale_x_continuous(
    labels = scales::comma,
    position = "top",
    expand = expansion(mult = c(0, 0.25))
  ) +

  # Apply the exact same manual alpha scale
  scale_alpha_manual(values = c("FALSE" = 1, "TRUE" = 0.4)) +

  facet_grid(geo_region ~ ., scales = "free_y", space = "free_y") +
  labs(title = "Export Value (USD Millions)", x = NULL, y = NULL) +
  theme_minimal() +
  theme(
    legend.position = "none",
    axis.text.y = element_blank(),
    panel.grid.major.y = element_blank(),
    panel.grid.minor = element_blank(),
    strip.text = element_blank(),
    plot.title = element_text(hjust = 0.5, face = "bold", size = 10)
  )
}
```

Save this chat

What is the difference between the two plots?

Gemini may display inaccurate info, including about people, so double-check its responses.

Because you set `legend.position = "none"` in the individual plot themes, `ggplot2` won't automatically try to build an ugly new legend for the `TRUE/FALSE` alpha mapping. It will simply apply the `0.4` opacity to your "Rest of" rows behind the scenes, creating a brilliant visual hierarchy w/ regional aggregates fall s

I want to address that the redesign uses the raw 32 categories instead of the aggregated AoA HS Proxy value that the original visualization uses, how should I phrase it?



To frame this as a major methodological improvement, you want to shift the focus from what you *did* ("we used raw outputs") to the **value it adds** for the audience.

By bypassing the generic `AoA HS proxy` aggregate and mapping out the 32 raw categories, you have unlocked granularity, allowed for precise unit filtering (like removing the USD categories), and brought out regional data profiles that an aggregate inherently hides.

Here are a few ways to phrase it, depending on the tone you want for your report or presentation:

Option 1: Professional & Impactful (Best for formal reports)

"Unlike the original visualization, which relies on the aggregated *AoA HS Proxy* values, this redesign utilizes the raw 32 product categories. By breaking down the data to its foundational level, we eliminate mixed-unit distortion and expose the granular, region-specific trade profiles that aggregated summaries inherently obscure."

Option 2: Technical & Methodological (Best for a methodology section)

"To ensure data integrity and granularity, the redesign bypasses the *AoA HS Proxy* aggregate layer in favor of the raw 32 category outputs. This structural shift enables us to isolate and filter out non-tonnage metrics (such as USD-denominated live animal and fruit categories), resulting in a mathematically precise volume-versus-value analysis."

Save this chat

Option 3: Concise & Direct (Best for slide decks or captions)

"Data Granularity: Shifted from the original chart's aggregated *AoA HS Proxy* value to the raw 32 category outputs. This allows for precise unit filtering and reveals true regional export dynamics."

Key themes you are hitting here:

Accuracy: You explicitly removed the mixed units (USD millions vs. Tonnes) that the proxy likely muddled together.

Granularity: You are showing the *actual* underlying data.

Transparency: You are showing exactly what makes up the numbers rather than hiding them behind a black-box "proxy" label.

Keep it short, this is on a poster

Perfect for a poster context—short, punchy, and structured for quick reading:

Data Methodology Update

Granular Insight: Shifted from the original chart's aggregated *AoA HS Proxy* to the raw 32 category outputs.

Higher Accuracy: Bypassing the aggregate allowed for precise unit filtering (removing USD-denominated products) to ensure a mathematically true volume vs. value comparison.

Hidden Patterns Unlocked: Revealing the raw data exposes unique regional export dynamics previously hidden behind a single proxy value.

Save this chat
Having AoA HS Proxy and 32 category outputs may seem

Gemini may display inaccurate info, including about people, so double-check its responses.

On a poster, you want to strip away the technical jargon ("AoA HS Proxy", "32 categories") so the general audience instantly gets the *why* without tripping over the naming conventions.

Here is a streamlined, user-friendly version tailored for a poster:

Methodology Improvement

From Summary to Detail: Shifted from the original chart's broad overview metric to the raw, unaggregated data.

Cleaner Unit Matching: Bypassing the summary metric allowed us to filter out mixed units (like separating dollar values from actual weights) for a true volume-versus-value comparison.

Hidden Trends Revealed: Accessing the raw data exposes specific regional trade dynamics that were previously masked by a single broad proxy.

I like it to be something as short as this but it still seems vague as our values are completely different from the original visualization because of this

▼

Here is a revision that keeps it incredibly brief for a poster but explicitly explains *why* the numbers have changed completely:

Methodology Shift

From Proxy to Raw Data: Bypassed the original chart's broad summary metric to rebuild the data directly from the raw, unaggregated commodities.

Why the Values Differ: Re-aggregating from the raw data allowed us to isolate and strip out mismatched units (like dollar-denominated goods), providing a mathematically accurate weight-vs-value comparison for the first time.

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.

Is it still too vague?

It borders on being a bit academic, which can read as vague if someone is just glancing at a poster. If your main goal is to instantly answer the viewer's question—*"Wait, why are these numbers completely different from the original chart?"*—you should make that the actual focal point.

Here are a few ultra-direct ways to phrase it, depending on the layout of your poster:

Option 1: The Direct Question (Best for a call-out box)

Why do our totals differ from the original?

We rebuilt the totals from the ground up using raw commodity data. This allowed us to remove incompatible units (like dollar-values mixed into tonnage) that were hidden within the original chart's broad summary metric.

[Google Privacy Policy](#)
[Google Terms of Service](#)
[Your privacy and Gemini Apps](#)

Option 2: The Two-Bullet Punch (Best for a Methodology section)

Methodology Update: Re-calculating the Totals

Save this chat

Gemini may display inaccurate info, including about people, so double-check its responses.